

2.1.1 Architecture

In our software, command-line applications are treated as the clients for the microservices. Microservices, databases and Kafka cluster, in conjunction, are treated as the servers. The client sends a request, in the form of JSON, over to the server by interacting with the implemented microservice's HTTP API with HTTP paths. The microservice then processes and persists or fetched the necessary data to and from its database. A response is sent back to the client, which is then outputted to the user. Kafka is used as a message broker between the microservices. Kafka events and streams, organised by topics, are published to and consumed from the Kafka clusters. Microservices consuming or subscribing to these events will be prompted to carry out their corresponding actions.

Microservices can run independently from each other in this architecture as they are not sharing the same resources to process data. This also means that individual microservices are scaled independently, and more resources can be devoted to the microservices with higher user demands. If there are user demands for a new service, the architecture, shown below, can scale by adding new microservices or building on the current microservices for new software features. A new database is required for every new microservices, and therefore, they can be easily integrated without affecting the overall architecture. Ports are required to be configured for the new microservices. New microservice is required to consume, subscribe and publish data to and from the right topics from the Kafka clusters, as some of the current event topics are strategically used to synchronise datum across all the microservices for the cohesion of the software.

Keys for the C4 diagram shown in the next page:

Topic A:

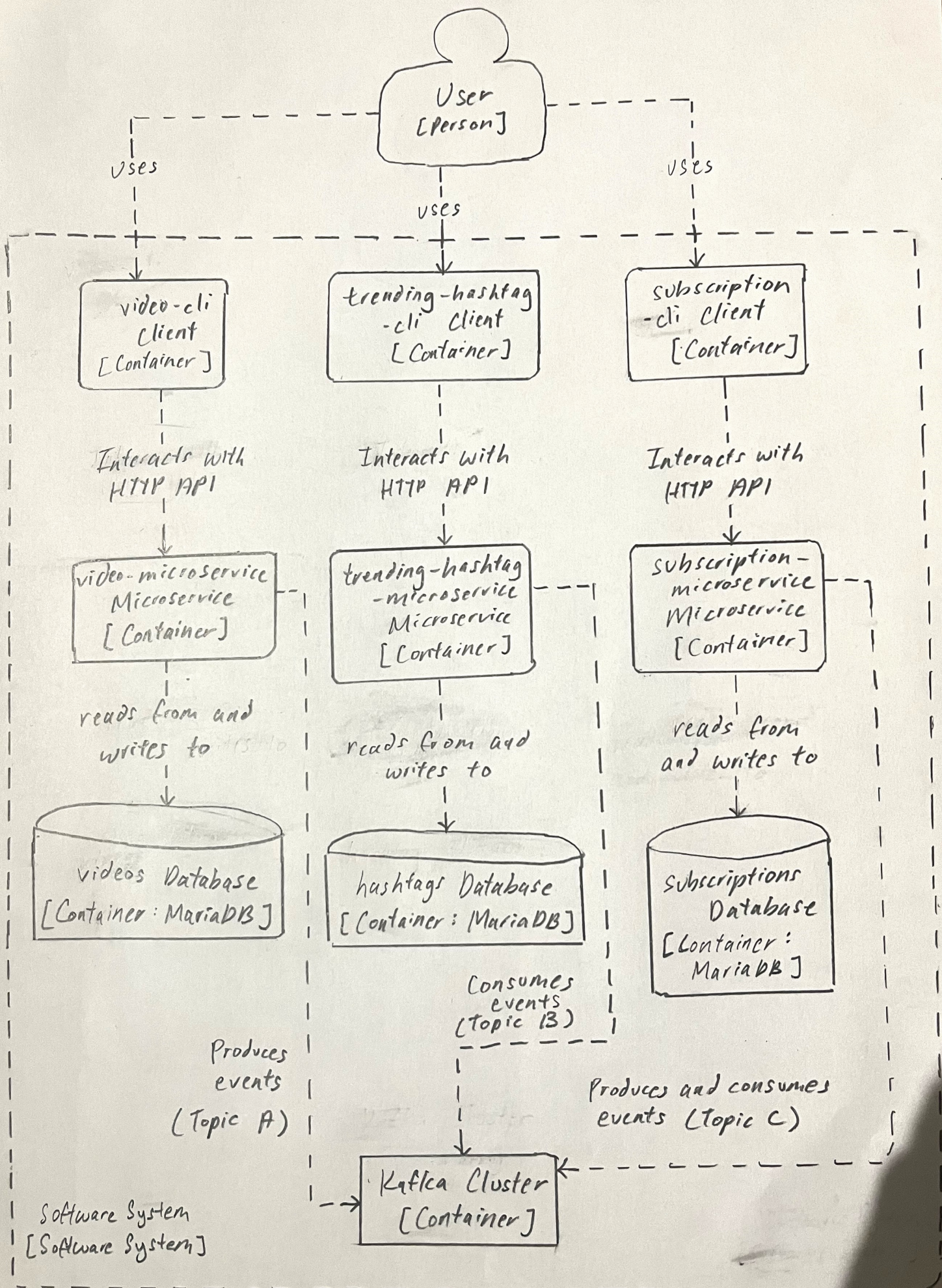
Topic produced
create-user
delete-user
post-video
update-video
delete-video
like-video
remove-like
create-hashtag
add-hashtag-video
remove-hashtag-video
delete-hashtag
top-liked-hashtags
dislike-video
remove-dislike
watch-video

Topic C:

Topic Produced	Topic consumed
subscribe-hashtag	create-user
unsubscribe-hashtag	delete-user
	create-video
	update-video
	delete-video
	like-video
	remove-like
	create-hashtag
	add-hashtag-video
	remove-hashtag-video
	delete-hashtag

Topic B:

Topic consumed / subscribed
top-liked-hashtags
delete-hashtag



2.1.2 Microservices

Clients

Video-cli commands:

create-user [name]	Create user with name <i>name</i>
get-users	Get all existing users
get-user [u_id]	Get user with id <i>u_id</i>
update-user [u_id] [n_name]	Update the name of the user with id <i>u_id</i> to a new name <i>n_name</i>
delete-user [u_id]	Delete user with id <i>u_id</i>
create-video [u_id] [title] -ht [hashtag_1] [hashtag_2] ...	Create video with title <i>title</i> and hashtag(s) <i>hashtag_*</i> by user with id <i>u_id</i> (Option <i>-ht</i> is used for the hashtag)
get-videos	Get all existing videos
get-video [v_id]	Get video with id <i>v_id</i>
update-video [u_id] [v_id] -t [title] -ht [hashtag_1] [hashtag_2] ...	Update by user with id <i>u_id</i> the title and hashtag(s) of the video with id <i>v_id</i> to a new title <i>title</i> and hashtag(s) <i>hashtag_*</i> (Option <i>-t</i> is used for the title and <i>-ht</i> for the hashtags) (This command takes no effect if the user is not the owner of the video)
delete-video [u_id] [v_id]	Delete video with id <i>v_id</i> by user with id <i>u_id</i> (This command takes no effect if the user is not the owner of the video)
add-view [v_id] [u_id]	Add a view to the video with id <i>v_id</i> , when it is watched by user with id <i>u_id</i>
add-like [v_id] [u_id]	Add a like by user with id <i>u_id</i> to the video with id <i>v_id</i> (Triggers add-view command) (Triggers remove-dislike command if the video is recently disliked by the user)
add-dislike [v_id] [u_id]	Add a dislike by user with id <i>u_id</i> to the video with id <i>v_id</i> (Triggers add-view command) (Triggers remove-like command if the video is recently liked by the user)
remove-like [v_id] [u_id]	Remove like by user with id <i>u_id</i> from the video with id <i>v_id</i>
remove-dislike [v_id] [u_id]	Remove dislike by user with id <i>u_id</i> from the video with id <i>v_id</i>
get-viewers [v_id]	Get all the users who liked the video with id <i>v_id</i>
get-likes [v_id]	Get all the users who liked the video with id <i>v_id</i>
get-dislikes [v_id]	Get all the users who disliked the video with id <i>v_id</i>
get-uploads [u_id]	Get all the video uploads for the user with id <i>u_id</i>
get-viewed-videos [u_id]	Get all the watched videos for the user with id <i>u_id</i>
get-liked-videos [u_id]	Get all the liked videos for the user with id <i>u_id</i>
get-disliked-videos [u_id]	Get all the disliked videos for the user with id <i>u_id</i>
get-hashtags	Get all existing hashtags
get-hashtag [h_id]	Get hashtag with id <i>h_id</i>
get-hashtag-videos [h_id]	Get all videos the contain the hashtag with id <i>h_id</i>

Trending-hashtag-cli commands:

get-trending-hashtags	Get current top 10 liked hashtags within a rolling time window of 1 hour
-----------------------	--

Subscription-cli commands:

subscribe-hashtag [h_id] [u_id]	Subscribe by user with id <i>u_id</i> to hashtag with id <i>h_id</i>
unsubscribe-hashtag [h_id] [u_id]	Unsubscribe by user with id <i>u_id</i> to hashtag with id <i>h_id</i>
get-recommendations [h_id] [u_id]	Get top 10 video recommendations with hashtag with id <i>h_id</i> for user with <i>u_id</i> (This command takes no effect if the user is not subscribed to the hashtag and instead, null will be returned)

Kafka

All topics in our microservices are named after their functionalities to be intuitive and self-explanatory, especially for the developer.

Topic	Produced from	Consumed from
create-user	video-microservice	subscription-microservice
delete-user		
post-video		
update-video		
delete-video		
like-video		
remove-like		
create-hashtag		
add-hashtag-video		
remove-hashtag-video		
delete-hashtag		subscription-microservice trending-hashtag-microservice
top-liked-hashtags		
dislike-video		
remove-dislike		
watch-video	subscription-microservice	
subscribe-hashtag		
unsubscribe-hashtag		

In trending-hashtag-microservice, an event stream that has a publisher with the topic “trending” and a subscriber with the topic “top-liked-hashtags” is used.

Domain Repositories and their fields:

video-microservice:	<pre>User { Long id String name Set<Video> uploads Set<Video> likedVids Set<Video> dislikedVids Set<Video> views } Video { Long id User creator String title Set<Hashtag> hashtags Set<User> likes Set<User> dislikes Set<User> viewers } Hashtag { Long id String name Set<Video> videos }</pre>
trending-hashtag-microservice:	<pre>Hashtag {Long id, String name, Integer numOfLikes }</pre>
subscription-microservice:	<pre>User { Long id, Set<Hashtag> subscriptions} Video { Long id, String title, Integer likes = 0, Set<Hashtag> hashtags} Hashtag {Long id, String name, Set<User> subscribers, Set<Video> videos }</pre>

2.1.3 Containerisation

A software comprises many services to offer to the users. Related services can be classified into a microservice application. Applications can be easily added to and removed from the software if needed without affecting other (running) applications, through the use of different ports and separate databases for each microservices. Referring to the “docker-compose.yml” file, video-microservice, trending-hashtag-microservice and subscription-microservice are assigned to ports 8080:8080, 8081:8080 and 8082:8080, and use the databases “videos”, “hashtags” and “subscriptions” respectively. Since the applications are not sharing the same resources, user data is only a concern to the microservices that the user is interacting with, meaning user traffic is managed within that microservice application. This, as a result, improves software efficiency and performance and makes it easier to scale up to larger numbers of users.

Another benefit of using this approach is that each application or feature of the software can be tested and deployed into the production environment independently of other applications. For example, each microservice has its unit testing, which will be discussed in the next section, and can generate its own image to be used by the container to run the application. Same features or applications can be recycled and integrated into different applications or software if required. For example, all three of our microservices are running from the same Kafka images/servers. The approach discussed above is an example of using isolation to be resilient to failures. For example, in our case, video-microservice and subscription-microservice are working properly, despite trending-hashtag-microservice has failed to carry out one of the operations, Kafka stream, required for the microservice to work as intended.

2.1.4 Quality Assurance

Unit Testing for Microservices

All tests implemented in all microservices have successfully passed with 100% success rate with an overall instruction coverage of 89% in video-microservice, 95% in trending-hashtag-microservice and 97% in subscription-microservice. Refer to the screenshots of the reports generated after running the microservice tests:

video-microservice

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
eng2.videos.controllers		90%		69%	41	106	28	272	1	30	0	3
eng2.videos.domain		73%	n/a		8	35	16	51	8	35	0	3
eng2.videos		0%	n/a		2	2	3	3	2	2	1	1
eng2.videos.dto		100%	n/a		0	8	0	11	0	8	0	2
Total	171 of 1,589	89%	46 of 152	69%	51	151	47	337	11	75	1	9

Package	Tests	Failures	Ignored	Duration	Success rate
eng2.videos	48	0	0	7.995s	100%

trending-hashtag-microservice

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
eng2.trending.hashtags		0%	n/a		2	2	3	3	2	2	1	1
eng2.trending.hashtags.events		100%		100%	0	8	0	22	0	7	0	2
eng2.trending.hashtags.controllers		100%		100%	0	5	0	10	0	4	0	1
eng2.trending.hashtags.domain		100%	n/a		0	7	0	10	0	7	0	1
Total	8 of 172	95%	0 of 4	100%	2	22	3	45	2	20	1	5

Package	Tests	Failures	Ignored	Duration	Success rate
eng2.trending.hashtags	8	0	0	1.818s	100%

subscription-microservice

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
eng2.subscriptions.domain		86%	n/a		3	23	5	34	3	23	0	3
eng2.subscriptions		97%		88%	5	21	3	50	2	8	1	2
eng2.subscriptions.events		100%		100%	0	17	0	57	0	13	0	2
Total	19 of 634	97%	3 of 34	91%	8	61	8	141	5	44	1	7

Package	Tests	Failures	Ignored	Duration	Success rate
eng2.subscriptions	17	0	0	5.040s	100%

Best Practice used for implementations

Using video-microservice as an example, and since there are only three domains, i.e. User, Video and Hashtag, all unit tests are categorized into three TestClasses, i.e. UsersControllerTest, VideosControllerTest and HashtagControllerTest, as shown in the screenshot below. Each test classes is responsible to run the tests rigorously only for its targeted Domain with fewer crossovers, thus improving code quality, and easier to locate bugs when debugging.

Class	Tests	Failures	Ignored	Duration	Success rate
HashtagsControllerTest	6	0	0	4.540s	100%
UsersControllerTest	25	0	0	1.888s	100%
VideoMicroserviceTest	1	0	0	0.008s	100%
VideosControllerTest	16	0	0	1.559s	100%

Using Test Cases

Different test cases are used on the same resources to ensure the resources work as intended under different conditions. For instance, as shown in the screenshots below, the resource deleteVideo from UsersController consists of three cases, i.e. expected inputs, creator (user) that does not exist or video that does not exist and user's permission to delete the video. Refer to Figure 3 for the test cases for deleteVideo.

```
@Delete("/{creator_id}/videos/{video_id}")
@Transactional
public HttpResponseMessage deleteVideo(Long creator_id, Long video_id){
    Optional<User> user = usersRepo.findById(creator_id);
    if (user.isEmpty()) {
        return HttpResponseMessage.notFound(String.format("User %d not found!", creator_id));
    }

    Optional<Video> video = videosRepo.findById(video_id);
    if (video.isEmpty()) {
        return HttpResponseMessage.notFound(String.format("Video %d not found!", video_id));
    }

    Video v = video.get();
    if (!v.getCreator().equals(user.get())) {
        return HttpResponseMessage.notFound(String.format("User %d has no permission to delete Video %d", creator_id, video_id));
    }
}
```

deleteVideo()	0.074s	passed
deleteVideoNotFound()	0.073s	passed
deleteVideoPermission()	0.102s	passed

Integration Testing

Using the conventions discussed above, we can fulfil the completeness and correctness of all the tests implemented. To expand the completeness and increase the correctness of the tests, integration testing can be used to verify that the microservices can communicate with the clients sufficiently, hence, reinforcing the robustness of the software. Refer to the screenshots below for the reports generated after running the client tests on top of the microservices.

Package	Tests	Failures	Ignored	Duration	Success rate
eng2.videos.cli	7	0	0	7.327s	100%

Class	Tests	Failures	Ignored	Duration	Success rate
CRUDTest	3	0	0	5.572s	100%
UserVideoInteractionsTest	3	0	0	1.616s	100%
VideoCliCommandTest	1	0	0	0.139s	100%

Package	Tests	Failures	Ignored	Duration	Success rate
eng2.trending_hashtags.cli	2	0	0	2.579s	100%

Package	Tests	Failures	Ignored	Duration	Success rate
eng2.subscriptions.cli	2	0	0	2.774s	100%

Due to large number of tests and limited amount of space in this report, for further test details, please refer to the reports generated under build/reports directory in the source folder of the microservices and clients.

2.2.1 Metamodel

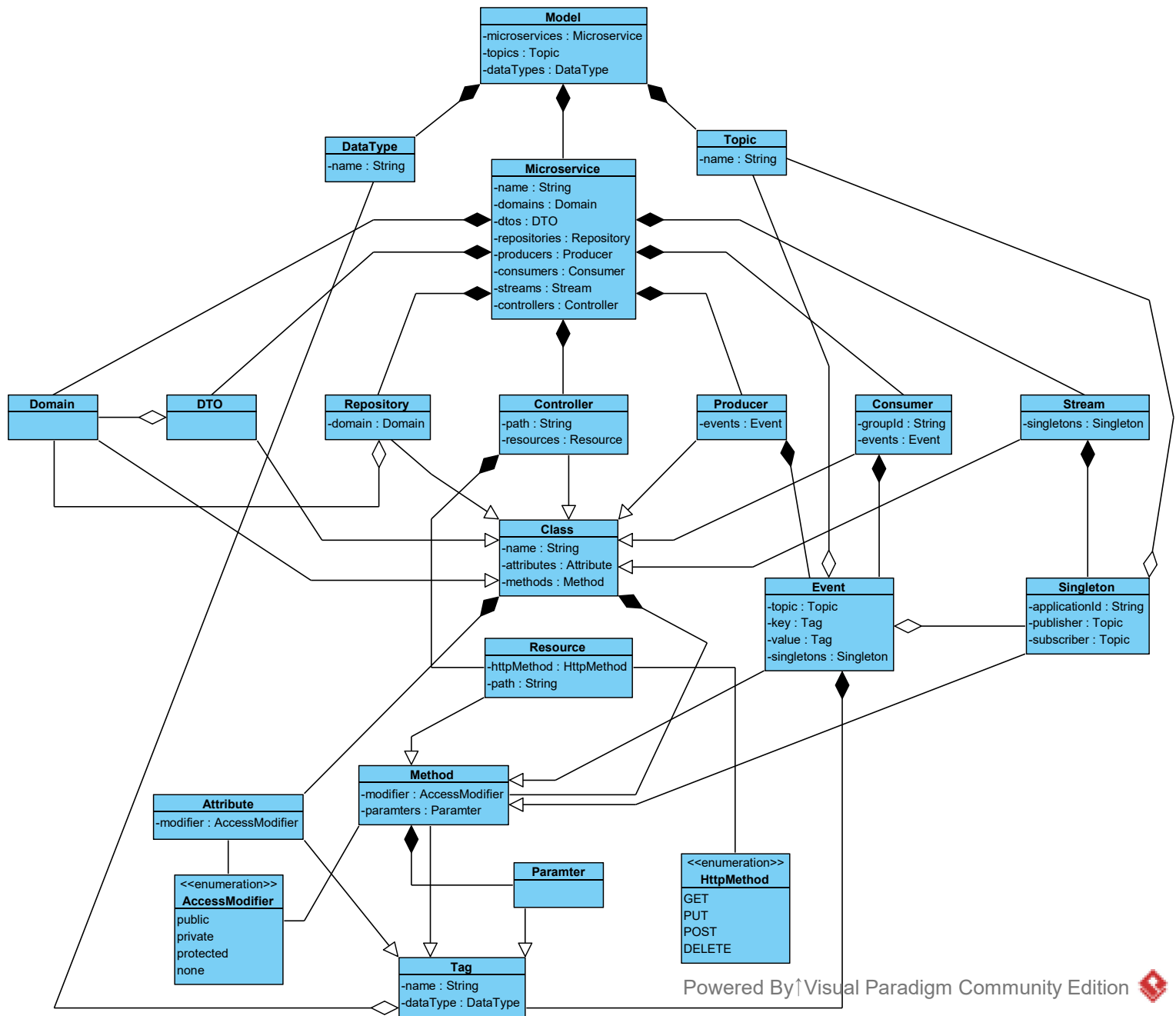
In the high level of abstraction, components in the metamodel are implemented to imitate the structure and basic characteristics of the components, i.e. Domain, DTO, Repository, Producer, Consumer, Stream and Controller, in the project that make up a microservice. Every class that represents a component in the metamodel inherits the Class class. A Stream Class must have at least one singleton (event stream), and each Singleton has a publisher and subscriber. A Producer or Consumer Class must have at least one event and each Event must have a topic, key and value. A Controller Class must have a path starting with "/" and at least one Resource. Each Resource must have a HTTP method and a path starting with "/". There are four HTTP methods, GET, POST, PUT, DELETE, and they are implemented as literals for HttpMethod Enum in the metamodel. The Method class is inherited by method variants such as Singleton, Event and Resource classes in the metamodel.

In the lower level of abstraction, the rest of the classes in the metamodel are implemented to imitate the characteristics of Java syntax. According to Java syntax, a Class has a name, might have attributes and methods. Each Attribute has a name, its datatype and might have an access modifier. Each Method has a name, its return type, might have parameters and an access modifier. There are only four access modifiers in Java, private, public, protected and default (none), and they are implemented as literals for AccessModifier Enum in the metamodel.

Since DataType and Topic objects are shared across different microservices, they are placed in Model class which is one abstraction layer higher than the Microservice class, and by doing this, same data type and topics can be directly referenced by the microservices that require them.

Inheritance and abstraction are used throughout the metamodel. Inheritance encourages code repeatability and reusability while abstraction helps with simplification of the code and protects data integrity. They are important as they reduce complexity and improve quality of the code.

Please refer to the class diagram of the metamodel in the next page.



2.2.3 Model Validation

Statement 1 : *“Every event stream needs to have at least one publisher and one subscriber.”*

Rationale:

An event stream is fully dependent on having at least one of the assigned publisher and subscriber. All event streams are unique, meaning that the event stream would not have existed without the assigned publisher or subscriber, i.e. the definition of the event stream will not be the same if its publisher or subscriber are changed, hence, the former event stream will be deleted and the new stream will be created.

Implementation:

There are two invariants for Event Stream. The constraint keyword is used for both invariants because they capture the strong consistency requirement mentioned in the statement and rationale.

The first invariant, named “atLeastOnePublisher”, has a condition that checks if the number of publishers assigned to the event stream is greater than or equal to 1. If the condition is false, the message “Stream needs to have at least one publisher!” will be prompted to notify the developer.

The second invariant, named “atLeastOneSubscriber”, has a condition that checks if the number of subscribers assigned to the event stream is greater than or equal to 1. If the condition is false, the message “Stream needs to have at least one subscribers!” will be prompted to notify the developer.

Note : In the actual EVL, event stream is represented by “Singleton” and Stream Class is represented by “Stream”.

Statement 2 : *“Every microservice needs at least one “health” resource using the HTTP GET method and taking no parameters, for reporting if it is working correctly.”*

Rationale:

Statement 2 ensures that every microservice has the basic resource to check if the HTTP API or the communication between client and server is working correctly. The resource is useful for testing and debugging. Having this convention improves overall time efficiency while testing and debugging.

Implementation:

The constraint keyword is used on the invariant because it captures the strong consistency requirement mentioned in the statement rationale. The invariant, named “atLeastOneHealthResource”, has a condition that checks if any of the controllers in the microservice contains a resource that has no parameters and its HTTP method is Get. If the condition is false, the message “Microservice needs at least one “health” resource!” will be prompted to notify the developer.

2.2.4 Model-to-Text Transformation

The EGX program and the microservice model are configured to interact with each other and generate code automatically based on the EGL templates for the model. The configuration file can be found under “modelling/configs” directory. The model is constructed upon the meta-data defined in the EMF. Each of the rules implemented in the EGX file takes in its corresponding EGL files as a parameter. Generated implementations of the model can be found under the target directory locations initialised by the rules. In our case, they are generated into the “src-gen” folder, which is in the directory for each microservices.

The generated implementations are written based off the basic structures of the seven elective components that made up the microservice, i.e. Domain, Repository, DTO, Producer, Consumer, Stream and Controller, along with the descriptions for each microservices that are mapped from the model into the placeholders in EGL templates. Placeholders in templates can be dynamically reused for different elements and minimises code repetitions. Having code generated automatically also means that the developer will have fewer code to write manually. This encourages code repeatability and reusability, hence increasing efficiency and performance of the software and developer. Once the code is generated, the organisation of the generated code shown below can be used as references to be compared to and justified by the actual code that is manually written in the “src” folder for each microservices. So far, the basic structures and characteristics of the generated and actual code match.

Domain:

```
[packages]
@Entity
@Serdeable
public class [name] {
    [attributes]
    [getters & setters]
}
```

DTO:

```
[packages]
@Serdeable
public class [name] {
    [attributes]
    [getters & setters]
}
```

Producer:

```
[packages]
@KafkaClient
public interface [name] {
    [events]
}
```

Repository:

```
[packages]
@Repository
public interface [name] extends CrudRepository<[domain.name],Long>{
}
```

Consumer:

```
[packages]
@KafkaListener(groupId="[groupId]")
public class [name] {
    [attributes]
    [events]
    [methods]
}
```

Stream:

```
[packages]
@Factory
public class [name] {
    [attributes]
    [singletons]
    [methods]
}
```

Controller:

```
[packages]
@Controller("[path]")
public class [name] {
    [attributes]
    [resources]
    [methods]
}
```

Attribute

```
private [dataType.name] [name];
```

Getter

```
public [attribute.dataType.name] get[attribute.name]() {  
    return [attribute.name];  
}
```

Setter

```
public void set[attribute.name]([attribute.dataType.name] [attribute.name]) {  
    this.[attribute.name] = [attribute.name];  
}
```

Method

```
private [dataType.name] [name]([paramters]) {  
}
```

Event in Producer

```
@Topic("[topic.name]")  
public void [name](@KafkaKey [key.dataType.name] [key.name], [value.dataType.name] [value.name]);
```

Event in Consumer

```
@Topic("[topic.name]")  
public void [name](@KafkaKey [key.dataType.name] [key.name], [value.dataType.name] [value.name]) {  
}
```

Singleton

```
@Singleton  
public [dataType.name] [name](ConfiguredStreamBuilder builder){  
}
```

Resource

```
@[httpMethod]("[path]")  
public [dataType.name] [name]([paramters]) {  
}
```

Notes:

The above organisations are just representations and reasonings of the actual EGL templates.

Square brackets represent placeholders that can be used dynamically by different elements of the same field.

0 or more elements can be present in the placeholders that have their label ending with "s".